

Task 3.1P

Exercise 1

1a Solution)

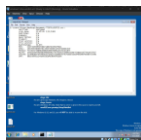
From my research, I found that Dyre (also called Dyreza or Dyzap) is a banking Trojan that was first discovered around June 2014. It was spread through phishing emails that tricked people into opening malicious attachments or clicking dodgy links. These emails usually carried a downloader called Upatre, which would then install Dyre onto the victim's machine in the background. Once installed, Dyre hooks into web browsers like Chrome, Firefox, and Internet Explorer to intercept banking credentials. It basically sits between the user and their bank's website, stealing login details as they type them in using man-in-the-browser attacks and fake login pages. The stolen data gets sent back to the attacker's command and control servers. What stood out to me is how sophisticated it is – it uses SSL encryption for its communications and has a modular design, which means the attackers can update its capabilities over time.

1b Solution)

A polymorphic virus is basically malware that changes how it looks every time it copies itself, while still doing the same malicious thing underneath. It might re-encrypt itself with a different key or shuffle parts of its code around, so each copy has a different binary signature. This is a big deal because traditional antivirus relies on matching known signatures to catch threats – if the signature keeps changing, the antivirus can't keep up. Dyre is considered polymorphic because every time someone downloads it from the distribution server, the file is slightly different. I read that Proofpoint researchers compared files downloaded just seconds apart from the same URL and found that about 10 bytes were changed at different offsets each time. That tiny difference was enough to give each file a completely unique hash, which meant hash-based blacklisting couldn't detect it. I thought that was really clever (in a scary way).

1c Solution)

Based on the Week 2 lecture categories, Dyre is a Trojan – specifically a banking Trojan. Trojans disguise themselves as legitimate software to trick you into running them. I could see this firsthand in the task because the Dyre file was named "Document-772976_829712.scr" and had a PDF icon, which made it look like a normal document. But when I opened it in the hex editor, the MZ header revealed it was actually an executable. So it's a textbook Trojan – it pretends to be something harmless to get the user to run it.



1d Solution)

Dyre's payload is stealing banking credentials. Once it's running on your system, it hooks into your web browser and waits for you to visit a banking website. When you try to log in, it intercepts your username and password using man-in-the-browser attacks and fake login pages, then sends everything to the attacker's servers. The threat to the victim is pretty serious – the attackers can use those stolen credentials to make unauthorised wire transfers and drain bank accounts. On top of that, Dyre can also spread itself by sending phishing emails to people in the victim's contact list, so it keeps infecting more people.

Exercise 2

2a Solution)

Hexadecimal (or just “hex”) is a base-16 number system. It uses the digits 0–9 and then the letters A–F to represent values 10–15. The reason it's so common in computing is that each hex digit maps to exactly 4 binary bits, so one byte (8 bits) can be neatly written as just two hex characters. For example, 11111111 in binary is FF in hex and 255 in decimal. This makes it way more compact than writing out long strings of 1s and 0s. I got to see this in action when I used the HxD hex editor to look at the Dyre file – the raw file content was displayed as rows of hex values, which is much easier to read than binary would be.

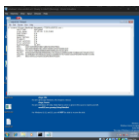
2b Solution)

The MZ header is the bytes 4D 5A at the very start of a file, which spells out “MZ” in ASCII. This tells us the file is a Windows executable (.exe or .dll). The initials come from Mark Zbikowski, who was one of the people that designed the MS-DOS executable format. When I opened the “Document-772976_829712.scr” file in HxD, the first thing I noticed was those 4D 5A bytes right at the beginning. That immediately told me this wasn't actually a PDF like the filename suggested – it was a disguised executable. I checked the Wikipedia list of file signatures and confirmed that 4D 5A corresponds to DOS MZ executables and Windows PE files. It's a pretty sneaky social engineering trick to make the file look like a harmless document when it's really malware.

Exercise 3

3a Solution)

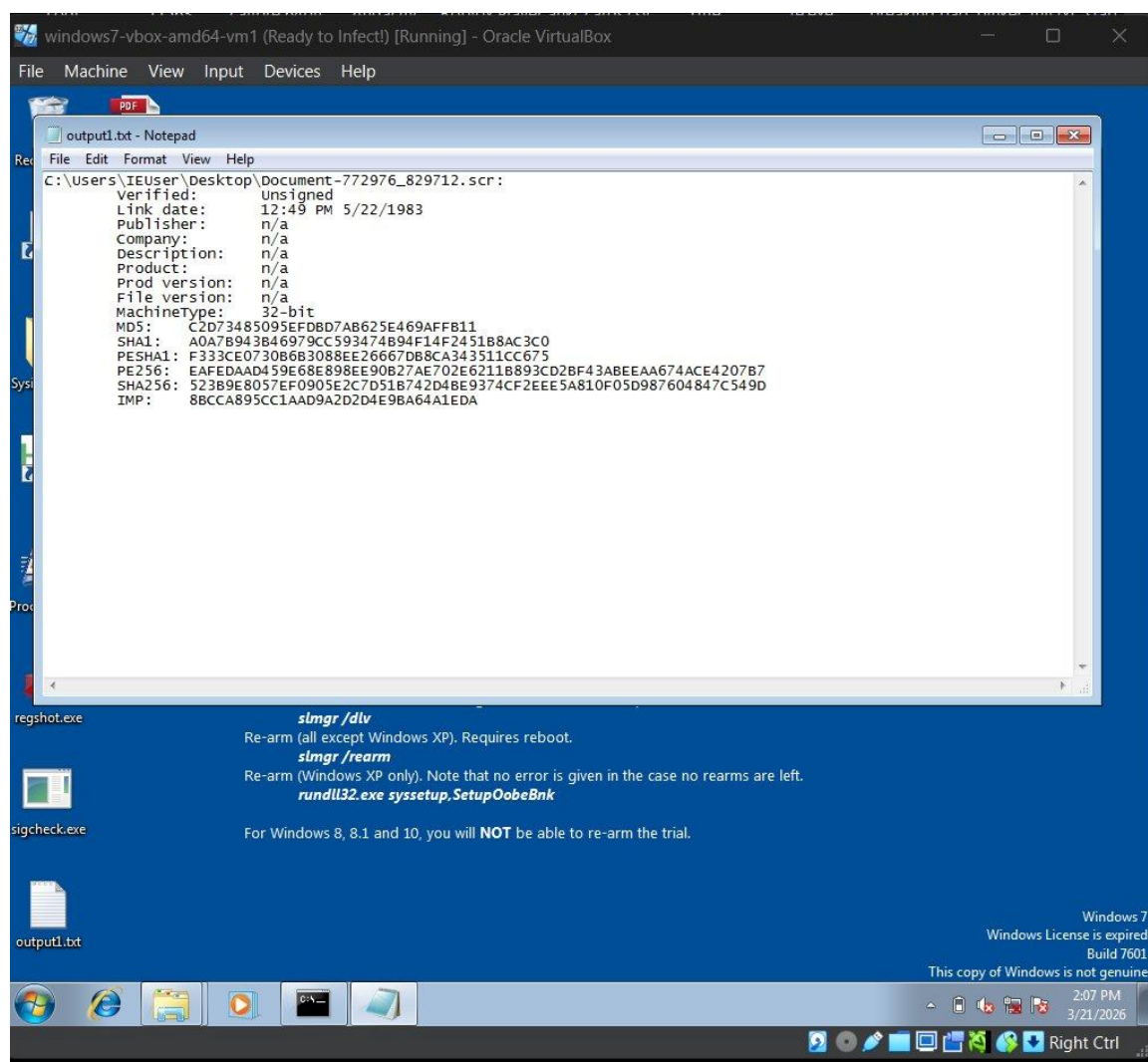
Sigcheck.exe is a command-line tool from Microsoft's Sysinternals suite. It checks files and shows you information like whether they're digitally signed, what version they are, and most importantly for this task, their cryptographic hash values. I used the -h flag which tells it to compute hashes like MD5, SHA1, and SHA256. When I ran “sigcheck.exe



-h Document-772976_829712.scr > output1.txt” it calculated all the hashes for the Dyre file and saved them to a text file. These hashes are like a unique fingerprint for the file – you can use them to look up the file on databases like VirusTotal to see if it’s known malware.

3b Solution)

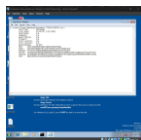
See screenshot below:



Exercise 4

4a Solution)

Looking at this through what we learned in Week 2’s lecture about malware behaviour, Dyre creates the second process (googleupdaterr.exe) as a way to stay hidden and persistent on the system. If the original file gets detected and deleted, the second copy keeps running. I noticed it uses the name “googleupdaterr.exe” which looks almost identical to a real Google update service – you’d probably scroll right past it in Task



Manager without thinking twice. That's the whole point, it blends in with legitimate Windows processes.

4b Solution)

The second process is basically Dyre's way of setting up shop permanently. It registers itself as a Windows service with a name like "Google Update Service" so it starts automatically every time the computer boots up. From there, it injects its code into legitimate system processes like `svchost.exe` or `explorer.exe`. Since those are trusted Windows processes, security software doesn't flag them. Once it's injected into those processes, it can hook into web browsers and start intercepting banking credentials without anyone noticing. So the second process is really where the actual credential theft happens – the first file just sets everything up.

Exercise 5

5a Solution)

From looking through the Procmon filtered results, I found that `googleupdaterr.exe` was stored in the user's local application data folder. The path was `C:\Users\IEUser\AppData\Local\googleupdaterr.exe`. It makes sense that the malware would hide itself in AppData rather than somewhere obvious like the Desktop, since most users never look in that folder.

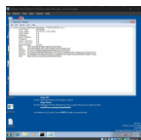
5b Solution)

When I compared the hashes in `output2.txt` with `output1.txt`, they were identical – the MD5, SHA1, and SHA256 values all matched. This means `googleupdaterr.exe` is literally an exact copy of the original Dyre file, just with a different name and stored in a different location. So what Dyre does when you run it is copy itself to the AppData folder under a name that looks like a legit Google service, and then the original file on the Desktop disappears. The content hasn't changed at all, it's the same malware just wearing a different disguise. I thought this was a good example of how malware tries to hide in plain sight.

Exercise 6

6a Solution)

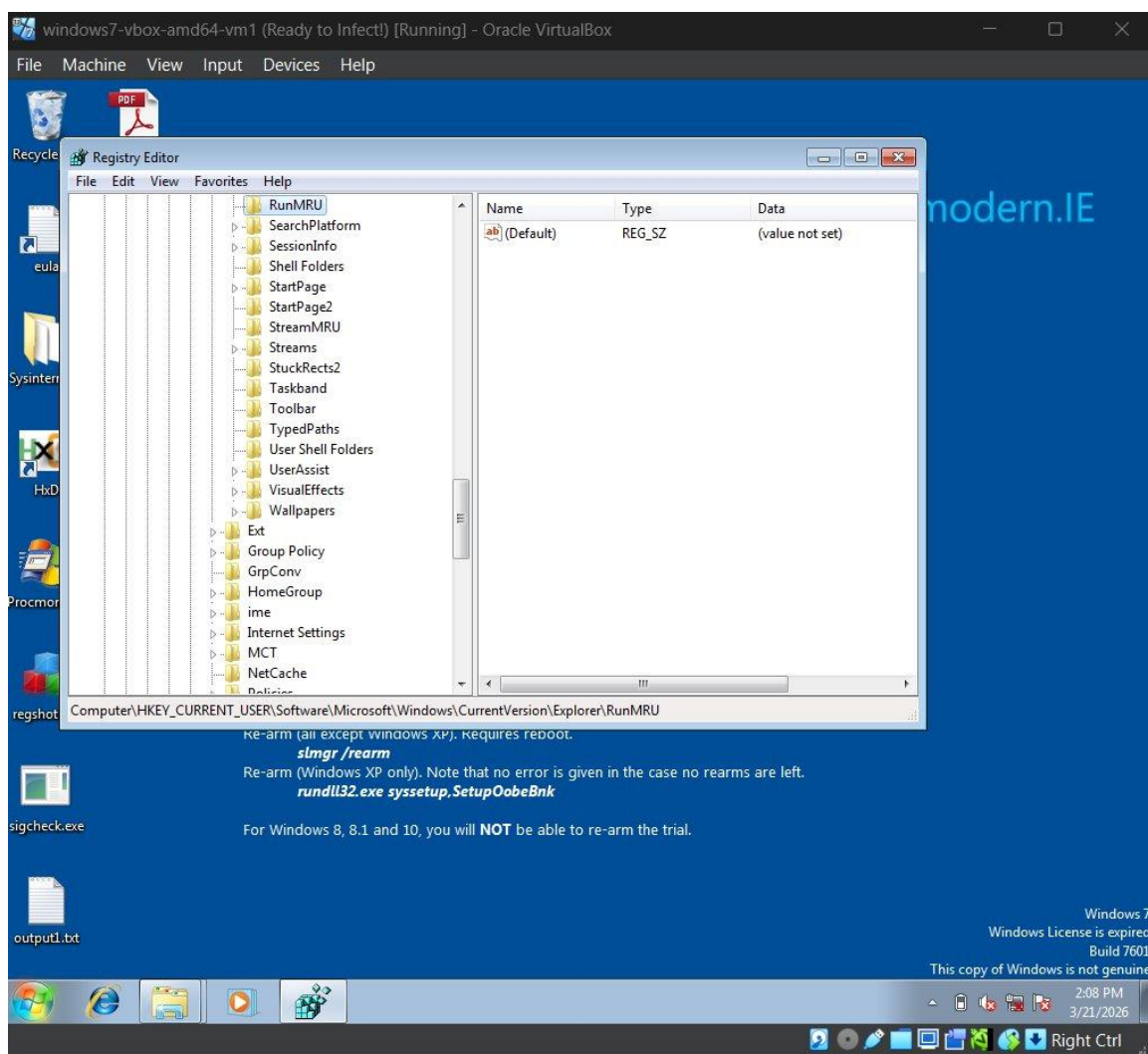
The Windows Registry is basically a big database built into Windows that stores all kinds of settings and configuration data. It holds settings for the OS itself, hardware, installed programs, and user preferences. When I opened it in the VM, I could see it's organised as a tree structure with main root keys like `HKEY_LOCAL_MACHINE` and



HKEY_CURRENT_USER, each containing subkeys and values. What's relevant to this task is that malware often targets the Registry to make itself persistent – by adding a registry entry that tells Windows to run the malware every time the computer starts up. That's exactly what Dyre does with its “Google Update Service” entry.

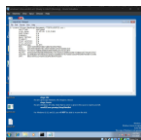
6b Solution)

To open the Windows Registry in Windows 7, you click the Start menu, type “regedit” in the search box, and press Enter. This launches the Registry Editor (regedit.exe), which provides a graphical interface to browse and edit registry keys and values. See screenshot below:



6c Solution)

The task involved both static and dynamic analysis, which we covered in Week 2's lecture. The static parts were when I examined the file in the hex editor (HxD) to look at the MZ header and when I used sigcheck to get the file's hash values – neither of those required actually running the malware. The dynamic parts were when I double-clicked the .scr file to execute it and then used Procmon to watch what it did in real time, and



when I used Regshot to compare the registry before and after infection. Most of the task was dynamic analysis since we actually ran the malware and observed its live behaviour, but the static analysis steps at the beginning were important too because they helped us confirm the file was an executable before we even ran it.

Exercise 7

7a Solution)

The Sophos article was about the Ragnar Locker ransomware gang, and honestly the way they pulled off their attack was pretty wild. After breaking into a company's network and getting admin access, they used Windows Group Policy to deploy a 280 MB Windows XP virtual machine along with VirtualBox onto the victim's system. They then configured the VM so it could access all of the host's drives – local disks, USB storage, network drives, everything. Then they ran their ransomware (only 49 KB) inside the VM, which encrypted all the host's files through those shared drives. The clever part is that from the host's perspective, the encryption was just VirtualBox's VboxHeadless.exe doing its thing, which looks like normal legitimate software. So the security software on the host had no idea ransomware was running. It basically flips the whole idea of virtual machines on its head – normally a VM protects the host from a dangerous guest, but here the attackers used the VM to protect their malware from the host's security tools.

7b Solution)

This task really brought the Week 2 lecture content to life for me. In the lecture we talked about Trojans, polymorphic viruses, and the difference between static and dynamic analysis, but it was all pretty abstract. Actually doing it myself made everything click. Opening the file in HxD and seeing the MZ header right there was the moment I understood how file signatures work – it was obvious straight away that this “PDF” was actually an executable. Running sigcheck to get the hashes was cool because I could see how analysts use those fingerprints to identify malware. The most interesting part for me was the dynamic analysis. When I ran the malware and watched Procmon capture everything in real time, I could see exactly how Dyre creates its googleupdaterr.exe copy and tries to hide itself. Using Regshot to see the registry changes also showed me why malware targets the registry for persistence. Comparing the hashes between the original and the copy and seeing they were identical really drove home how the malware operates. One challenge I had was getting the sigcheck command right – I typed the filename wrong the first time and got an empty output, which taught me to use Tab autocomplete. Overall I feel much more confident about malware analysis concepts now compared to just reading about them in the lecture.